VHDL | recto · le langage

mot-clé type intégré -- commentaire «à remplir»

Bibliothèques

1 · structure

```
library ieee;
use ieee.std_logic_1164.all;  -- std_logic, rising_edge
use ieee.numeric_std.all;     -- unsigned, signed, +, -
use work.outils.all;          -- un paquet du projet
```

⚠Piège : jamais `std_logic_arith` ni `std_logic_unsigned` : non standard, en conflit avec `numeric_std`.

</> Entité : l'interface

```
entity compteur is
  generic (
    N    : positive := 8;      -- paramètre
    MAXV : natural  := 255
  );
  port (
    clk, rst : in  std_logic;
    en       : in  std_logic;
    q        : out std_logic_vector(N-1 downto 0);
    pleine   : out std_logic   -- pas de ; final
  );
end entity;
```

`in`	lecture seule
`out`	écriture (lisible en interne en VHDL-2008)
`inout`	bidirectionnel (bus trois états)
`buffer`	sortie relisible (à éviter)

Ports et génériques sont séparés par des `;`, pas des virgules.

</> Architecture : le contenu

```
architecture rtl of compteur is
  -- zone déclarative : signaux, constantes, types
  signal cnt    : unsigned(N-1 downto 0);
  signal data_b : std_logic_vector(7 downto 0);
  constant ZERO : unsigned(cnt'range) := (others => '0');
begin
  -- instructions concurrentes (ordre indifférent)
  q      <= std_logic_vector(cnt);
  pleine <= '1' when cnt = MAXV else '0';
end architecture;
```

Plusieurs architectures par entité possibles ; `rtl`, `behav` sont des noms libres.

Paquet et fonction

```
package outils is        -- déclarations
  constant LARG : positive := 32;
  function maxi(a, b : integer) return integer;
end package;
package body outils is  -- définitions
  function maxi(a, b : integer) return integer is
  begin
    if a > b then return a; else return b; end if;
  end function;
end package body;
```

Une fonction rend une valeur, sans **wait**. Une **procedure** peut attendre (**wait**) et a des paramètres **out**.

std_logic : 9 valeurs

2 · types · littéraux

`'U'`	non initialisé	début de simulation
`'X'`	conflit fort	deux sources opposées
`'0'` `'1'`	niveaux forts	valeurs logiques
`'Z'`	flottant	haute impédance (trois états)
`'W'`	conflit faible	
`'L'` `'H'`	0 et 1 faibles	pull-down, pull-up
`'-'`	indifférent	optimisation de synthèse

Type résolu : plusieurs pilotes se combinent (`'0'` et `'1'` donnent `'X'`).

Types

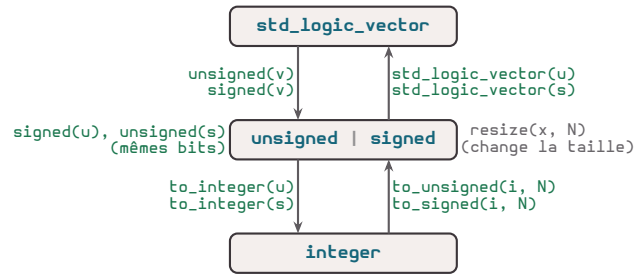
`std_logic_vector`	bus de bits, sans valeur numérique
`unsigned`, `signed`	bus avec arithmétique (`numeric_std`)
`integer range 0 to 9`	32 bits si pas de `range`
`natural`, `positive`	entiers ≥ 0, ≥ 1
`boolean`	résultat des comparaisons
`time`	ns, us… (simulation)

```
type etat_t is (IDLE, LIRE, ECRIRE);  -- énuméré (FSM)
subtype octet is std_logic_vector(7 downto 0);
type mem_t is array (0 to 255)
  of std_logic_vector(7 downto 0);    -- tableau
type pix_t is record
  r, g, b : unsigned(7 downto 0);
end record;
signal mem : mem_t := (others => (others => '0'));
```

</> Littéraux et agrégats

```
'0' '1' 'Z'                -- std_logic : apostrophes
"1010"   x"FF"   o"17"     -- vecteurs : guillemets
16#FF#   2#1010#   1_000_000  -- entiers dans une base
10 ns    1.5 us            -- time : espace obligatoire
(others => '0')            -- tout à 0, toute taille
(0 => '1', others => '0')  -- bit 0 à 1
a & b       v(7 downto 4)  -- concaténation, tranche
```

Conversions (numeric_std)




```
y <= std_logic_vector(unsigned(a) + unsigned(b));
n <= to_integer(unsigned(sel));  -- index
r <= resize(s, 16);              -- signe étendu
```

</> Opérateurs

3 · opérateurs · instructions

Arithm.	`+ - * / **` `mod rem abs`	sur `unsigned`, `signed`, `integer` ; `mod` du signe du diviseur, `rem` du dividende
Logiques	`and or not xor`	aussi `nand nor xnor` ; bit à bit sur vecteurs
Relation	`= /= < <= > >=`	`/=` différent ; rendent un `boolean`
Décalage	`shift_left(u, 2)`	`shift_right` étend le signe sur `signed`
Concat.	`a & b`	assemble des vecteurs
Affect.	`<=` / `:=`	signal / variable

⚠Piège : `and` et `or` ont la même priorité : `a and b or c` est refusé, parenthéser.

Instructions concurrentes

```
y <= "0001" when a = "00" else  -- conditionnelle
     "0010" when a = "01" else
     "0100" when a = "10" else
     "1000";                    -- else final
with sel select                 -- sélective
  m <= x0 when "00",
       x1 when "01",
       x2 when others;
g : for i in 0 to N-1 generate  -- répliquer
  y(i) <= a(i) xor b(i);
end generate;
```

Process : le séquentiel

```
process (clk, rst)                  -- liste de sensibilité
  variable tmp : integer := 0;      -- locale au process
begin
  if rst = '1' then
    ...
  elsif rising_edge(clk) then
    case op is
      when "00"         => tmp := tmp + 1;
      when "01" | "10"  => tmp := 0;
      when others       => null;    -- others obligatoire
    end case;
    for i in v'range loop r(i) <= v(7 - i); end loop;
  end if;
end process;
```

Signal : nouvelle valeur à la fin du process ; variable `:=` : tout de suite.

Attributs

`v'length`	nombre d'éléments
`v'range`	`7 downto 0` : pour `for i in v'range`
`v'high`, `v'low`	bornes
`v'left`, `v'right`	bornes dans l'ordre déclaré
`T'image(x)`	texte de x (`integer'image(n)`)
`clk'event`	front : préférer `rising_edge(clk)`

VHDL — verso · conception et simulation

mot-clé type intégré -- commentaire «à remplir»

Logique combinatoire

4 · circuits

```
process (all)  -- VHDL-2008 : tout lu
begin
  y <= '0';    -- défaut : pas de latch
  case sel is
    when "00"   => y <= a;
    when "01"   => y <= b;
    when others => null;
  end case;
end process;
-- additionneur complet (flot de données)
sum  <= a xor b xor cin;
cout <= (a and b) or (cin and (a xor b));
```

Avant 2008 : mettre dans la liste de sensibilité tous les signaux lus, sinon simulation et synthèse divergent.

Trois états

```
b   <= a when oe = '1' else (others => 'Z');  -- sortie
din <= pad;                                  -- inout lu
```

Registre N bits

```
-- generic (N : positive := 16) ; d, q : N bits
process (clk, rst)    -- reset asynchrone
begin
  if rst = '1' then
    q <= (others => '0');
  elsif rising_edge(clk) then
    if en = '1' then  -- clock enable
      q <= d;
    end if;
  end if;
end process;
```

Reset synchrone : `process (clk)` et tester `rst` à l'intérieur de `rising_edge(clk)`.

Compteur

```
process (clk)
begin
  if rising_edge(clk) then
    if rst = '1' then       -- reset synchrone
      cnt <= (others => '0');
    elsif en = '1' then
      if cnt = MAXV then cnt <= (others => '0');
      else cnt <= cnt + 1;  -- unsigned + integer
      end if;
    end if;
  end if;
end process;
```

Décalage et détection de front

```
process (clk)
begin
  if rising_edge(clk) then
    sr   <= sr(6 downto 0) & din;  -- registre à décalage
    b_r  <= b;            -- b retardé d'un cycle
  end if;
end process;
front <= b and not b_r;  -- impulsion 0 -> 1
```

Instancier un composant

5 · hiérarchie · FSM

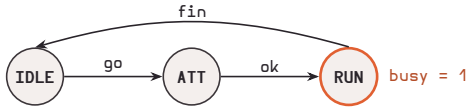
```
-- direct (VHDL-93) : pas de déclaration à faire
u_reg : entity work.reg
  generic map (N => 8)
  port map (clk => clk, rst => rst, en => '1',
            d => d_in, q => open);  -- open : non relié
-- classique : déclarer le composant, puis
component fa is port (a, b, cin : in  std_logic;
  sum, cout : out std_logic); end component;
f0 : fa port map (a(0), b(0), cin, s(0), c(0));  -- ordre
```

Préférer l'association par nom (=>) : l'ordre des ports ne compte plus.

Répliquer : additionneur N bits (generate)

```
signal c : std_logic_vector(N downto 0);  -- retenues
c(0) <= cin;
g : for i in 0 to N-1 generate
  fa_i : entity work.fa
    port map (a => a(i), b => b(i), cin => c(i),
              sum => s(i), cout => c(i+1));
end generate;
cout <= c(N);                  -- + simple : Conversions
```

Machine à états (Moore)




```
type etat_t is (IDLE, ATT, RUN);
signal etat : etat_t;
process (clk)
begin
  if rising_edge(clk) then
    case etat is
      when IDLE => if go = '1' then etat <= ATT; end if;
      when ATT => if ok = '1' then etat <= RUN; end if;
      when RUN => if fin = '1' then etat <= IDLE; end if;
    end case;
    if rst = '1' then etat <= IDLE; end if;  -- l'emporte
  end if;
end process;
busy <= '1' when etat = RUN else '0';  -- sorties
```

RAM synchrone

```
type ram_t is array (0 to 2**AW - 1)
  of std_logic_vector(DW-1 downto 0);
signal ram : ram_t;
process (clk)
begin
  if rising_edge(clk) then
    if we = '1' then
      ram(to_integer(unsigned(adr))) <= din;
    end if;
    dout <= ram(to_integer(unsigned(adr)));  -- 1 cycle
  end if;
end process;                   -- reconnu comme BRAM
```

Testbench

6 · simulation

```
library ieee;  use ieee.std_logic_1164.all;
entity tb_and2 is end entity; -- pas de ports
architecture sim of tb_and2 is
  signal a, b, c : std_logic := '0';
  signal clk     : std_logic := '0';
begin
  clk <= not clk after 5 ns;  -- horloge 100 MHz
  uut : entity work.and2
    port map (a => a, b => b, c => c);
  stim : process
  begin
    a <= '1'; b <= '1';
    wait for 10 ns;
    assert c = '1' report "1 and 1 doit donner 1"
      severity error;
    wait until rising_edge(clk);
    report "fin" severity note;
    std.env.stop;              -- VHDL-2008 ; sinon wait;
  end process;
end architecture;
```

`wait for t`	attendre une durée
`wait until c`	attendre que c devienne vrai
`wait on s`	attendre un changement de s
`wait;`	suspendre pour toujours
`severity`	`note warning error failure` (arrêt)

Afficher des valeurs

```
report "cnt = " & integer'image(to_integer(cnt));
report "v = " & to_hstring(v);  -- VHDL-2008 (hexa)
report "t = " & time'image(now);
```

Lire des vecteurs dans un fichier (textio)

```
use std.textio.all;          -- < 2008 : + std_logic_textio
file f : text open read_mode is "stim.txt";
variable l : line;
variable v : std_logic_vector(7 downto 0);
while not endfile(f) loop
  readline(f, l);  read(l, v);    -- une ligne : 01011010
  a <= v;  wait until rising_edge(clk);
end loop;
```

Pièges

Signal / variable	lire un signal juste après l'avoir affecté rend l'ancienne valeur.
Latch	un `if` sans `else` dans un process combinatoire mémorise : mettre une valeur par défaut.
Pilotes multiples	deux process sur un signal : `'X'` si les valeurs s'opposent, refusé sur `integer`.
`if s then`	refusé si s est un `std_logic` : écrire `if s = '1' then` (`boolean` attendu, sauf VHDL-2008).
Mots réservés	`bus`, `in`, `out`, `range`… ne peuvent pas nommer un signal.
`downto` / `to`	`(7 downto 0)` : bit 7 à gauche (MSB) ; ne pas mélanger.
Init.	`signal s : ... := '0'` ne sert qu'en simulation (ou FPGA) : prévoir un reset.